kiln: a small Linux sandbox runtime, built layer by layer

A design paper

Stephen Estrada

github.com/lambda-nine/kiln

October 2026

Abstract

kiln is a container runtime of about 3,800 lines of Rust that runs commands in isolated Linux sandboxes, either one-shot or in persistent sandboxes that accept further commands. It was built to learn, from the kernel interfaces up, what a container actually is: seven namespaces, an overlay root filesystem swapped in with `pivot_root`, cgroup v2 limits on memory, CPU, processes and block I/O, XFS project quotas for disk, a user namespace that makes sandbox root an unprivileged host UID, a seccomp filter and a Landlock ruleset, a per-sandbox network namespace on a bridge with an nftables egress policy, a policy-enforcing DNS forwarder, and a stateless Prometheus exporter. Each layer was written directly against the kernel's files, system calls and netlink (with the `nft` tool for firewall rules), with no container libraries. On one ARM Lima VM, an idle-gap sweep put kiln and bare crun's one-shot `true` lifecycle at 14.9 and 14.8 ms after 5 s idle, against 140.9 ms for Docker. Across five fresh boots at a defined 100 ms gap, the paired geometric mean of Docker/kiln per-boot median ratios was 25.84× (95% session-bootstrap interval 23.82–27.82×). These are runtime-overhead measurements, not end-to-end agent speed. This paper describes each layer and the design decision behind it, where start-up time went, what two model-assisted adversarial reviews found, and what is still missing. kiln is experimental defense in depth for a trusted operator, not a hostile multi-tenant boundary: sandboxes share the host kernel and one host UID/GID range.

1 Motivation and goals

kiln is a learning project with a specific aim: real depth in Linux systems concepts (cgroups, namespaces, filesystems, networking, observability). The fastest way to get that depth is to build the thing that exercises it, so kiln re-implements the core of a container runtime: the part that `runc` and Docker hide. Its intended use is running tool calls for AI agents (shell commands, scripts, package installs) where the code is untrusted and its resource use unpredictable.

Goals. (1) Every isolation and limit mechanism is implemented by kiln itself against the kernel interface: `clone`, `mount`, cgroupfs files, `ioctl`s, raw BPF, `nft` rules. (2) Every limit has a measurable failure mode that was triggered on purpose (an OOM kill, a fork bomb, a full disk, a rejected packet). (3) No central sandbox-manager daemon and no database: core state lives in the kernel and a few directories, non-networked starts need no shared service, and the system can be inspected with `cat`, `nft list` and `findmnt`. (4) Defaults are closed: no network unless asked, limits on by default.

Non-goals (for now). OCI image and runtime-spec compatibility, rootless operation, multi-tenant production hardening, multi-host orchestration, and a stable public API (§17).

Environment. macOS has no namespaces or cgroups, so kiln runs in an Ubuntu 24.04 arm64 VM under Lima (4 vCPU, 4 GiB) with the source tree shared into the VM. The image is Alpine's minirootfs, fetched by a script that verifies its release signature.

2 Architecture overview

kiln is one binary. `kiln run` is the one-shot runtime: it creates a sandbox, supervises it, and tears it down. `kiln create`, `exec`, `ls`, `rm` and `gc` manage persistent sandboxes (§7). `kiln dns` and `kiln metrics` are long-running host services under systemd: a DNS forwarder for sandboxes and a Prometheus exporter. `kiln stats` prints a `docker stats`-style table. Figure 1 shows how they relate.

No central manager daemon, no database. A run's identity is its *run ID*, the start time in nanoseconds (hex) plus kiln's PID: unique, sortable, and used to name both its cgroup (`/sys/fs/cgroup/kiln/<id>`) and its run directory (`/var/lib/kiln/runs/<id>`). IP leases and DNS policies are files named after the sandbox's IP, and tools find running sandboxes by listing `/sys/fs/cgroup/kiln/`. The cost of this design is that recovery after a crash must be explicit: `kiln gc` and lease reconciliation (§7, §11) rebuild their view from these same files.

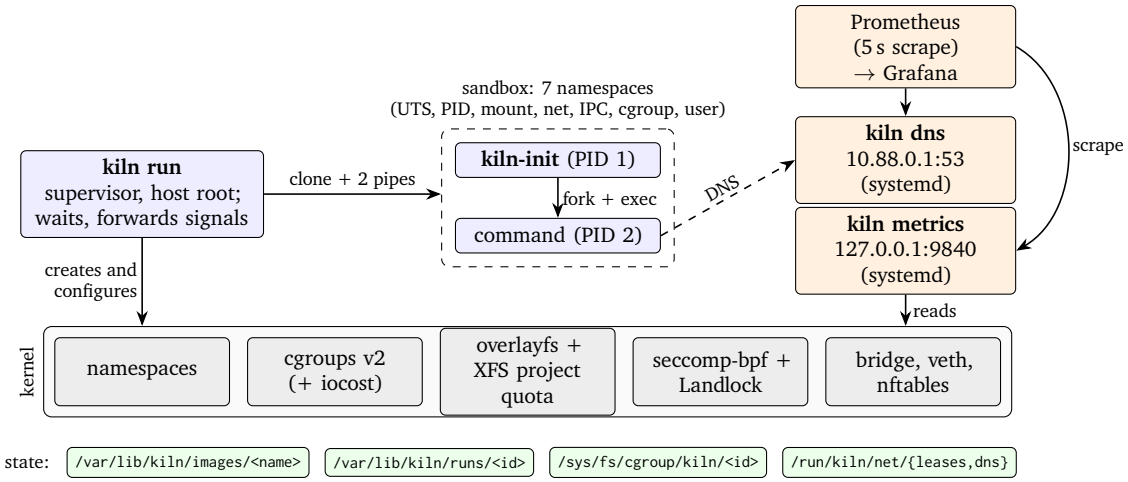


Figure 1: kiln's parts. The supervisor creates each sandbox and configures the kernel on its behalf; the two services run under systemd and are shared by all sandboxes. All state is kernel objects plus the four directories at the bottom.

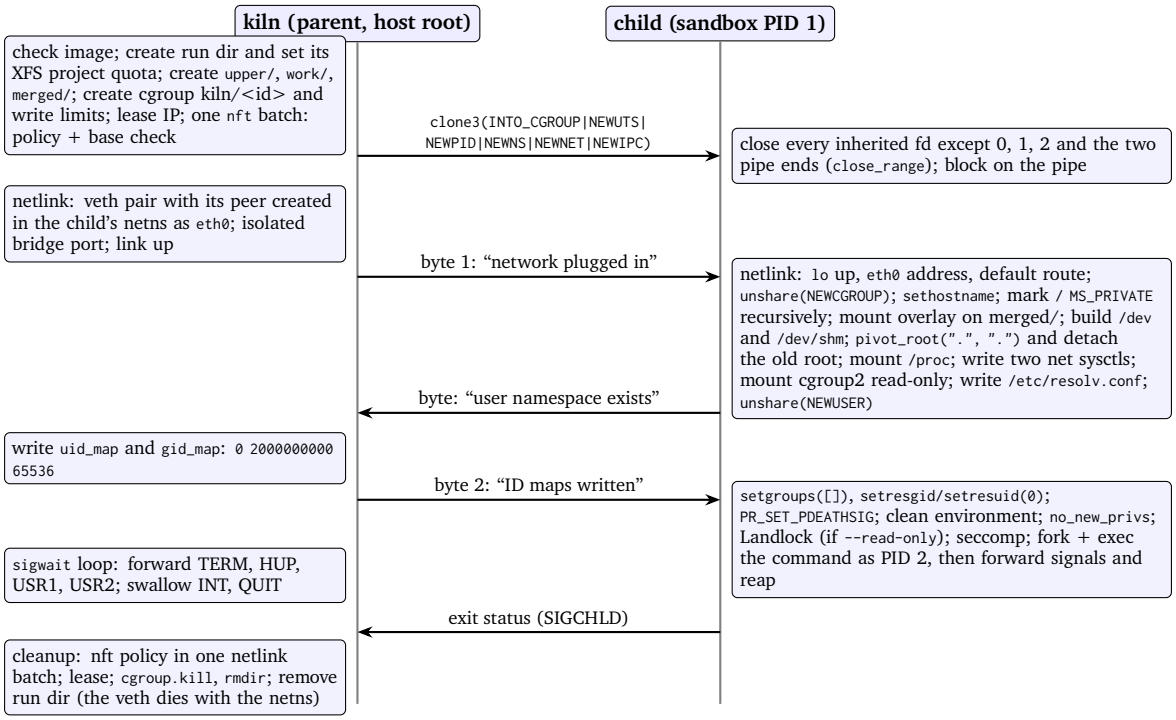


Figure 2: Sandbox startup. The child is born inside its cgroup; three one-byte messages over two `O_CLOEXEC` pipes order the steps that must happen outside the sandbox (network plumbing, ID maps) relative to those inside it. If either side dies, the other reads EOF instead of hanging.

Resource ownership through RAII. The supervisor holds a `Cgroup` value and an optional `Network` value whose destructors run on every exit path: the nftables policy and the lease, then `cgroup.kill` and `rmdir`, then the run directory (unless `--keep`). The overlay mount and the veth live in the sandbox's namespaces and disappear with them.

3 Process and namespace setup

A namespace gives a process its own instance of one kind of global kernel resource. kiln creates five at `clone3` time: UTS (hostname), PID (the child becomes PID 1 of a new numbering), mount, network, and IPC (System V shared memory, semaphores and message queues). The child later unshares two more: a cgroup namespace and, last of all, a user namespace. Without `CLONE_VM`, `clone3` behaves like `fork`: the child gets a copy-on-write copy of the parent's memory, configuration included.

Figure 2 is the whole startup protocol. Three ordering constraints shape it:

1. **Limits before the first instruction.** `clone3` with `CLONE_INTO_CGROUP` and an open fd of the cgroup directory creates

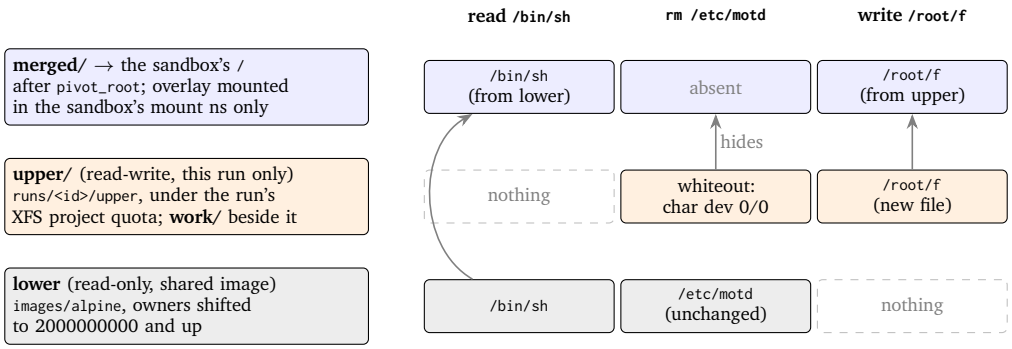


Figure 3: The overlay root. Reads fall through to the shared image; writes and deletions land only in the run's upper layer, so `upper/` is an exact record of what the sandbox changed. On top of `merged/`, the sandbox mounts its own `/proc`, a minimal `/dev` tmpfs, a 64 MiB `/dev/shm` and a read-only cgroup2 at `/sys/fs/cgroup`.

the child already inside it, so no instruction runs outside its limits. (The older way, writing the PID to `cgroup.procs` before releasing the child, was the slowest step of a start; §15.) The network is plugged in before the child is released.

2. **The cgroup namespace inside the cgroup.** A cgroup namespace makes the process's *current* cgroup appear as the root. `clone3` creates namespaces before placing the child, so the child unshares it itself; otherwise the host's cgroup would become the sandbox's root.
3. **The user namespace last.** Every namespace is owned by the user namespace that was current when it was created, and capabilities only act on namespaces that one's user namespace owns. Creating the user namespace after all the others leaves them owned by the host, so sandbox root, with a full capability set, cannot mount, unmount, change the hostname or reconfigure its network.

File descriptors are the first thing closed. Namespaces, `pivot_root` and Landlock all check *paths* at open time; none of them revokes a descriptor that is already open. A descriptor inherited from whatever started kiln is therefore a handle straight through every wall (§16). The child closes everything except stdio and its two pipe ends before doing anything else, which also means a parent that dies mid-setup shows up as EOF on the pipe.

4 Root filesystem: overlayfs and pivot_root

Every sandbox shares one read-only image and gets a private writable layer (Figure 3). overlayfs merges them: lookups check the upper layer first; the first write to a lower file copies it up; a deletion leaves a *whiteout* (a 0/0 character device) in the upper layer. Two consequences matter for agents. Destroying the image from inside is impossible (`rm -rf /bin` only writes whiteouts), and `upper/` is a precise, free audit log of the run's changes; `--keep` preserves it for inspection. Mount points that kiln itself needs (`/sys/fs/cgroup`, `/workspace`) are created in the image at fetch time, so `upper/` holds only what the sandbox wrote.

Swapping roots. `chroot` only changes how paths are resolved and leaves the old root reachable to a sufficiently privileged process. `pivot_root` changes the mount namespace's root mount itself. kiln uses the `pivot_root(".", ".")` idiom: with the current directory at the new root, the old root ends up stacked on top of it, and `umount2(".", MNT_DETACH)` then removes it, leaving no mount through which the host filesystem can be reached.

Mount propagation. On systemd hosts `/` is a *shared* mount, so even in a new mount namespace, new mounts would propagate back to the host, and `pivot_root` refuses to operate on shared mounts. The child first marks everything `MS_REC|MS_PRIVATE`. During a run, `findmnt -t overlay` on the host is empty.

A minimal /dev. Instead of the host's `/dev` (which includes disks), the sandbox gets a 64 KiB tmpfs holding bind mounts of six devices (`null`, `zero`, `full`, `random`, `urandom`, `tty`), the usual `fd` and `std*` symlinks, and a `/dev/shm` tmpfs whose pages are charged to the sandbox's memory limit. A fresh `/proc` (mounted `nosuid,nodev,noexec`) shows only the sandbox's PID namespace.

5 Resource limits with cgroups v2

cgroup v2 is a single hierarchy of directories; each directory is a group of processes, and controller files inside it set limits and report usage. Two rules shape kiln's layout (Figure 4). Controllers are enabled top-down: a cgroup can only use `memory`, `cpu`, `pids` or `io` if its parent lists them in `cgroup.subtree_control`. And a cgroup that delegates controllers to children cannot hold processes itself ("no internal processes"). So sandboxes live *under* `/sys/fs/cgroup/kiln/`, which

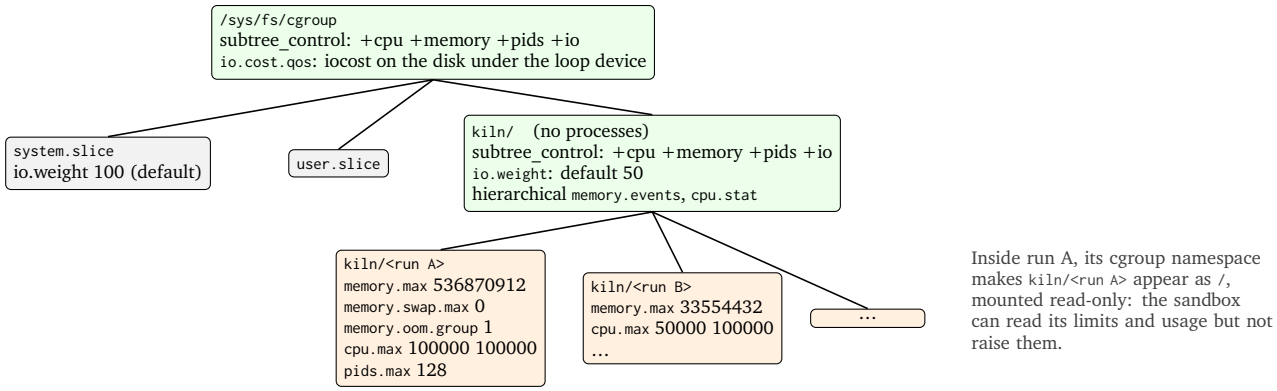


Figure 4: cgroup hierarchy. Limits are set per run; the parent sets the sandboxes' collective I/O share and provides counters that survive individual runs.

Table 1: Limits, their defaults, and the kernel interface behind each.

Flag	Default	Kernel interface	At the limit
`--memory`	512M	`memory.max`; `memory.swap.max`=0; `memory.oom.group`=1	reclaim, then the OOM killer kills every process in the sandbox
`--cpus`	1	`cpu.max` = "q 100000", q = cores × 100 000 µs	throttled for the rest of each 100 ms period
`--pids`	128	`pids.max`	`fork`/`clone` fail; counted in `pids.events`
`--disk`	1G	XFS project quota: block hard limit, plus one inode per 16 KiB (min. 128)	writes fail with `ENOSPC`
(all runs)	—	parent `io.weight` "default 50" with iocost	proportional share, only under contention
(all runs)	—	tmpfs `size=`: 64 KiB `/dev`, 64 MiB `/dev/shm`	`ENOSPC`; shm pages count toward `memory.max`

itself stays empty.

Table 1 lists every limit. The design decisions behind them:

- **Kill the sandbox, not a process.** `memory.oom.group` makes an OOM kill take out the whole group. A half-killed tool call (a shell whose child died) is harder to reason about than one that failed outright. Swap is disabled for the group (when a memory limit is set) so a memory hog hits its limit instead of degrading into swap.
- **CPU is a quota per period**, not a share: `--cpus 0.25` writes `25000 100000`, so a 2 s busy loop gets 500 ms of CPU.
- **Page cache is memory.** Writing 100 MB showed a memory peak of about 100 MiB: file pages are charged to the writer's cgroup. They are reclaimable, but they count toward `memory.max` (§13).
- **Teardown is atomic.** `cgroup.kill` SIGKILLs every process in the group in one write, including anything that left the command's process tree.

6 kiln-init: PID 1 and signals

The kernel treats a PID namespace's init specially: signals sent to it from inside its namespace are dropped unless it installed a handler, and orphaned processes are re-parented to it. A normal program as PID 1 therefore ignores `kill` and `timeout` and never reaps orphans. Before kiln-init, `kiln run -- timeout 2 sh -c 'while :; do :; done'` never ended: busybox `timeout` signalled from inside the namespace and PID 1 ignored it.

So PID 1 is kiln itself (the same binary, already running in the child), playing the role `tini` plays for Docker. It blocks all signals, forks the command as PID 2 (which restores an empty mask and `exec`s), and then loops on `sigwait`: on SIGCHLD it reaps *every* exited child (several exits can coalesce into one signal), and SIGTERM, SIGHUP, SIGUSR1 and SIGUSR2 are forwarded to the command. When the command exits, init returns its status (128 + signal number if it was killed, 127 if `exec` failed) and exits; the kernel then kills whatever is left in the PID namespace.

Waiting for signals rather than handling them. Both kiln-init and the supervisor block signals with `sigprocmask` and consume them synchronously with `sigwait` in the same loop that reaps children. There are no asynchronous handlers, so there are no async-signal-safety rules to get wrong. The supervisor therefore survives Ctrl-C and `kill` and still cleans up. SIGINT, SIGQUIT and SIGWINCH are not forwarded: the terminal already sends them to its whole foreground process group, which includes the sandboxed command.

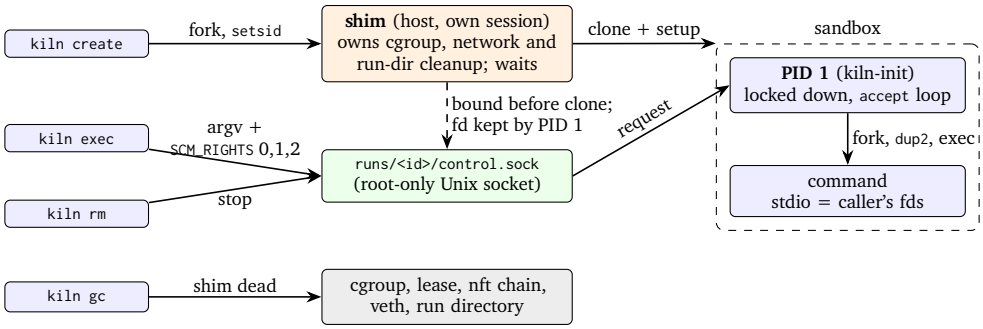


Figure 5: Persistent sandboxes. A detached shim owns each sandbox's host resources; PID 1 serves `exec` and `rm` requests on a socket opened before `pivot_root`. Exit codes travel back over the same connection. `kiln gc` removes what a dead shim left behind.

7 Persistent sandboxes

Agents need state between tool calls: a package installed by one command must be there for the next. `kiln create` starts a sandbox that outlives the command that created it and prints its ID; `kiln exec <id> -- <cmd>` runs a command in it; `kiln ls` lists sandboxes and whether their shim is alive; `kiln rm` stops one and waits until its resources are gone; and `kiln gc` cleans up after shims that died. One-shot `kiln run` is unchanged.

A shim per sandbox. `kiln create` forks a host process, which starts its own session, detaches from the terminal, and does exactly what the one-shot supervisor does (run directory and quota, cgroup, `clone`, network, ID maps), then reports ready over a pipe so the CLI can print the ID and exit. The shim stays alive as the sandbox's parent, holding the `Cgroup` and `Network` values whose destructors clean up. This is containerd's shim pattern: a sandbox's lifetime is tied to a small process dedicated to it, not to the CLI or a future daemon. If the shim is killed, `PR_SET_PDEATHSIG` takes PID 1 down, and `kiln gc` finds run directories whose shim is dead and removes their cgroup, network state and files (Figure 5).

Exec through PID 1, not `setns`. The usual design is for the host to enter the sandbox's namespaces with `setns`, as `docker exec` does through runc. A process entering from outside arrives without the sandbox's credentials, cgroup, Landlock ruleset or seccomp filter, and each must be re-applied in the right order: a second code path for every ordering constraint of §3. kiln makes the sandbox's own PID 1 the exec server instead. Before cloning, the shim binds a Unix socket in the run directory. The child keeps that one descriptor when it closes all others, completes the full lockdown, sends a third handshake byte, and loops on `accept`. `kiln exec` connects, sends the argument vector as a length-prefixed message, and passes redirected stdin, stdout and stderr as `SCM_RIGHTS` descriptors. It refuses terminal descriptors until kiln has a controlled PTY mode. PID 1 forks, duplicates them onto 0–2, and `execs`: the command inherits every namespace, limit, filter and rule simply by being PID 1's child. PID 1 reaps it and writes the exit status back, so `kiln exec` exits with the command's own code.

Consequences. After lockdown, the only host objects the sandbox can reach are the listening socket (whose path is not visible inside) and the descriptors a caller chose to pass. Requests are size-bounded and parsed strictly, and run one at a time. Files persist in the upper layer. Running `true` this way takes 3.5 ms (p50, one client, uncontended), against 9.7 to 53.7 ms for the other runtimes' `exec` (§15). `kiln rm` sends a stop request; PID 1 acknowledges and exits, the kernel kills the rest of the PID namespace, and the shim's normal cleanup runs.

8 Disk quotas with XFS project quotas

Without a disk limit, one sandbox can fill the host filesystem. Classic quotas limit what a UID owns, which does not fit (all sandboxes map to the same host IDs, §9). XFS *project* quotas instead limit whatever is tagged with a 32-bit project ID, and the `PROJINHERIT` flag on a directory makes everything created beneath it inherit the tag. This is how Docker's overlay2 driver limits container size.

Storage. `/var/lib/kiln/runs` is an XFS filesystem inside a sparse file (20 GB by default), mounted through a loop device with `prjquota`, so no spare partition is needed. Per run, kiln tags the still-empty run directory with `FS_IOC_FSSETXATTR` (project ID plus `PROJINHERIT`) and sets the limits with `quotactl_fd` (`Q_XSETQLIM`), both as raw `ioctl`/syscall calls against `#[repr(C)]` structs whose sizes are asserted at compile time. `upper/` and `work/` are created afterwards, so they and everything the sandbox writes are charged to the run.

Design decisions. (1) The project ID is the run directory's inode number: unique on the filesystem for as long as the directory exists, so no ID allocator is needed. (2) Because inode numbers are reused, kiln *always* writes the limit, even "unlimited" (0); otherwise a new run could inherit a dead run's limit. (3) Blocks alone are not enough: empty

files cost almost no blocks, so a small block quota could still exhaust the filesystem's inodes. Each project also gets an inode limit of one per 16 KiB of quota (at least 128). (4) A full quota surfaces to the program as `ENOSPC` ("No space left on device"), not `EDQUOT`, which is the honest description from inside.

9 User namespaces and ID mapping

Without a user namespace, root in the sandbox is root on the host, and every other wall becomes the last one. With it, sandbox UIDs and GIDs 0–65535 map to host IDs 2 000 000 000–2 000 065 535, a range no real user owns (well above the `/etc/subuid` range given to the VM's user). An escaped process lands on the host as UID 2 000 000 000, which owns nothing.

Only an outsider can write the maps. A fresh user namespace has no mapping, so its process appears as the overflow ID (`nobody`). The parent, which holds `CAP_SETUID/CAP_SETGID` over the parent namespace, writes `0 2000000000 65536` to the child's `uid_map` and `gid_map`; the child then drops supplementary groups and switches to UID/GID 0. That is the second pipe handshake in Figure 2.

Capabilities that apply to nothing. Inside, root has a full capability set (`CapEff: 000001ffffffffff`), but since the mount, UTS, network, IPC, PID and cgroup namespaces are owned by the host's user namespace, `hostname`, `mount`, `umount` and `ip link set` all fail with `EPERM`.

Image files must be shifted. Host-root-owned image files would appear as `nobody` inside, and sandbox root could not modify them. The fetch script therefore adds 2 000 000 000 to every file's owner and group, restoring setuid bits that `chown` clears (Docker's userns-remap approach), and kiln refuses to start if the image root is not owned by the base ID. The upper layer is chowned to the same ID so the sandbox's `/` belongs to sandbox root. The known cost is that all sandboxes share one ID range (§17).

Trusting the image. The image is the root filesystem of every sandbox, so the fetch script treats the download as hostile until proven otherwise. It accepts only a strictly formed `alpine-minirootfs-*-<arch>.tar.gz` name, checks the published SHA-256, and verifies the detached signature with `gpgv` against Alpine's release key, which is pinned in the repository by fingerprint. A checksum from the same server as the file proves only integrity; the signature proves who published it. The image is then unpacked and shifted in a staging directory on the same filesystem and swapped in with `renameat2(RENAME_EXCHANGE)`, so a sandbox starting during an update sees either the old image or the new one, never a half-written tree.

Because sandbox root does not own its network namespace, it could not bind ports below 1024 or ping; the child sets two per-namespace sysctls (`ip_unprivileged_port_start`, `ping_group_range`) to allow both, as Docker does.

10 Syscall filtering and Landlock

Namespaces control what a process can *see*; the kernel is still shared, and every system call is attack surface. Two final restrictions are applied just before the command starts, both inherited by every descendant and irrevocable.

no_new_privs comes first. It guarantees that `exec` can never grant more privilege (setuid bits stop working), which is the precondition for an unprivileged process to install a seccomp filter or Landlock ruleset: otherwise a filtered process could exec a setuid binary that runs with the filter but with more privilege.

seccomp-bpf. A seccomp filter is a small classic-BPF program the kernel runs on every syscall, over a struct holding the architecture, syscall number and arguments. kiln's is hand-assembled. It first checks the architecture, since syscall numbers differ per ABI, and kills the process on a mismatch (and, on x86-64, on x32 syscalls). It then denies, with `EPERM`, about 40 syscalls in a few families: loading kernel modules and kexec; large, historically bug-prone subsystems (`bpf`, `perf_event_open`, `userfaultfd`, `io_uring`); the kernel keyring, which is not namespaced; open-by-handle, which bypasses path checks; `setns` and the whole mount API; and host-wide state (clocks, swap, accounting, kernel log, quotas). Finally it denies any `clone` or `unshare` whose flags include a `CLONE_NEW*` bit. That matters because sandbox root holds every capability *inside* its user namespace: a new network or mount namespace it created would be its own, exposing root-only kernel code. `clone3` passes its flags in memory the filter cannot read, so it returns `ENOSYS`, which makes libc fall back to `clone` (the approach Docker uses). The filter is installed with `SECCOMP_FILTER_FLAG_TSYNC`, a no-op for today's single-threaded child but correct for a future multi-threaded one.

Decision: a denylist, readable in one sitting, rather than an allowlist like Docker's (∼350 syscalls); the cost is that new syscalls are allowed.

Landlock (with `--read-only`) restricts which paths may be written, and unlike file permissions it binds root (`CAP_DAC_OVERRIDE` does not apply). A ruleset declares the access rights it *handles*; any handled right is denied unless a rule grants it beneath some directory. kiln handles only write-type rights (writing, truncating, creating, removing

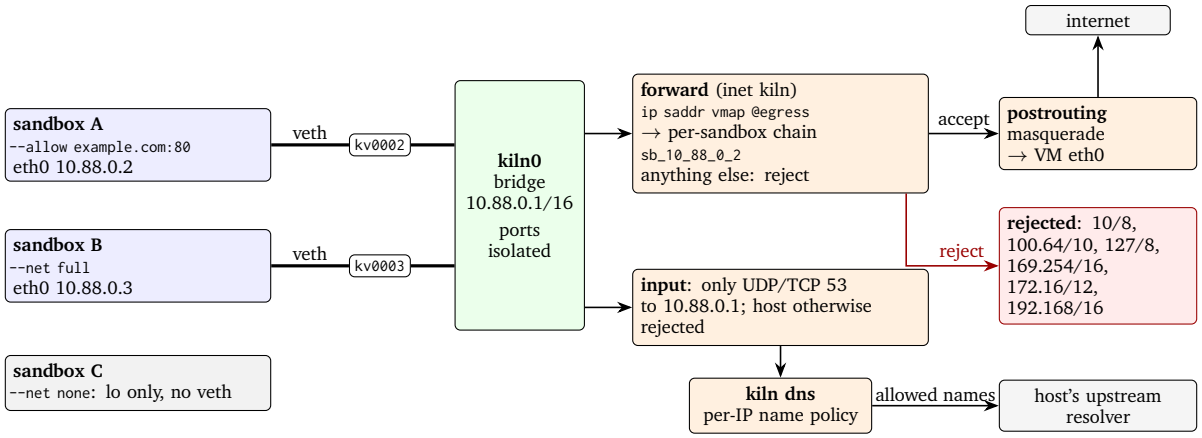


Figure 6: Network topology. Each networked sandbox has a veth pair whose host end is an isolated port on `kiln0`. All policy is in one nftables table: a verdict map sends each sandbox's packets to its own chain, the input chain admits only DNS to the forwarder, and NAT masquerades the rest out of the VM.

and renaming across directories), so reading and executing stay allowed everywhere, and grants them beneath `/tmp`, `/root`, `/workspace`, `/dev/shm` and any `--writable` paths; existing `/dev` files stay writable but no new devices can be created. kiln requires Landlock ABI 3 (Linux 6.2), the first that covers truncation; on older kernels `--read-only` would silently allow files to be emptied. Landlock does not cover metadata, so `chmod` on a file the sandbox owns still works.

11 Networking: bridge, veth pairs and nftables

Every sandbox gets its own network namespace. With `--net none` (the default) it contains only loopback. With `--net full` or `--allow`, the supervisor connects it to the host (Figure 6).

Plumbing from outside. The network namespace is created at `clone3` time, before the user namespace, so sandbox root cannot change its address, routes or firewall. The supervisor leases an address and installs the sandbox's nftables chain before the clone, then creates a veth pair whose peer is created directly inside the child's namespace as `eth0` and attaches the host end to `kiln0` as an isolated port. kiln-init, still real root and before its user namespace exists, brings up `lo`, assigns its address and adds a default route via 10.88.0.1. Both sides speak rtnetlink directly (a small hand-written client), the protocol `ip` uses, rather than starting `ip` or `nsenter`.

Shared state is reconciled, not just detected. The bridge and the base nftables objects are shared by all sandboxes, so every networked run checks them under one `flock`. The sandbox's policy batch ends with `list table`, and that one listing is compared with the canonical sets, map, counters and hook chains. Only drift is committed as a repair, in one transaction that leaves per-sandbox chains alone; a canonical ruleset (every run after the first) commits nothing.

Leases. A lease is a file in `/run/kiln/net/leases` created with `O_EXCL`, so concurrent runs cannot get the same address. A lease left by a killed supervisor is reclaimed only when *every* sign of its run is gone (cgroup, veth, nft map entry, live supervisor): any one of them is absent during part of a healthy setup. Leases on the `/run` tmpfs vanish on reboot.

Policy is one verdict map. The base table's forward chain is:

```
iifname "kiln0" ct state new add @connlimit { ip saddr ct count over 256 } \
    counter name "conntrack_rejected" counter reject with icmpx admin-prohibited
iifname "kiln0" ip saddr vmap @egress   # jump to this sandbox's chain
iifname "kiln0" counter reject with icmpx admin-prohibited
oifname "kiln0" ct state established,related accept
oifname "kiln0" counter drop
```

Adding a sandbox is one map element plus one chain; removing it deletes both, with no rule handles to track. For `--net full` the chain rejects the private and link-local set (the VM's LAN, the Mac behind it, cloud metadata at 169.254.169.254) and accepts the rest. For `--allow HOST[:PORT]` it accepts only the listed addresses (hostnames are resolved once, at start) and rejects everything else. The sandbox's source address is trustworthy because sandbox root cannot change it. Rules end in `reject` rather than `drop`, so a blocked connection fails in milliseconds instead of hanging until a timeout, which matters for an agent waiting on a tool call.

Bounding a host-wide table. The conntrack table is shared by the whole host; one sandbox opening many distinct flows could fill it. The first rule counts tracked connections per source IP in a dynamic set and rejects new flows beyond 256. Because IPs are reused, the IP's count is destroyed at setup and teardown (§16).

Isolation between sandboxes. Frames between two ports of one bridge are switched, not routed, so the inet forward

hook never sees them. kiln sets the bridge port flag `isolated on` instead, which stops isolated ports from talking to each other at layer 2. The input chain keeps sandboxes off the host's own services (`sshd`, and later kilnd's API).

12 The DNS forwarder

Name resolution was the last way out: DNS can carry data in query names (`<secret>.attacker.example`), and originally sandboxes could reach the VM's resolver directly. Now each networked sandbox's `/etc/resolv.conf` (written as real root, so read-only inside) points at `kiln dns` on 10.88.0.1:53, the only resolver the input chain lets it reach. For each query the forwarder reads the policy file named after the sender's IP: `*` for `--net full`, otherwise the `--allow` hostnames and their subdomains. Allowed queries go to the host's upstream resolver; others, and senders with no policy file, get `REFUSED`, so with `--allow` an attacker's domain is never resolved.

It parses only what it needs (the 12-byte header with exactly one question, then length-prefixed labels; compression pointers are rejected), accepts upstream replies only with a matching transaction ID, and turns upstream failures into `SERVFAIL`. Since sandboxes can send it unlimited traffic, 16 workers serve a queue of 256 jobs and drop on overflow (clients retry), and the systemd unit caps tasks and memory. Every query is logged with control characters escaped and counted for Prometheus. The bounds and the escaping came from the second review (§16).

13 Storage and I/O accounting

An OOM kill caused by someone else's cache. A sandbox writing 1.5 GB with `O_DIRECT` under a 512 MB limit was OOM-killed: the loop driver wrote the XFS image's backing file through the page cache, and those dirty pages were charged to the sandbox that caused the I/O. Direct I/O on the loop device (`LOOP_SET_DIRECT_IO`, checked on every run) removed the double caching; the same test peaked at 2.4 MiB. **Proportional I/O.** `io.weight` needs a proportional controller, and the VM's disk uses `mq-deadline` without BFQ, so kiln enables iocost on the real disk under the loop device and gives all sandboxes together half a host service's weight. Writes are accounted per sandbox in `io.stat`, but on this VM the weights do not change throughput: iocost enforces them only when its model judges the device saturated, which the Mac's SSD behind virtio never is under the default model. Accounting works; enforcement is unproven.

14 Metrics: Prometheus and Grafana

A stateless exporter. `kiln metrics` reads the kernel's own counters on every scrape: each sandbox cgroup's memory, CPU, PIDs, `io.stat` and pressure (PSI) files, its XFS project quota, the host end of its veth, and `nft list table inet kiln`, written in the Prometheus text format by hand. Prometheus scrapes every 5 s because most sandboxes live for seconds, and a provisioned Grafana dashboard shows the result.

Short runs fall between scrapes. A sandbox OOM-killed half a second after starting never appears in a scrape. The parent's `kiln/memory.events` and `kiln/cpu.stat` are hierarchical and keep counting after children are removed, so totals since boot include every run (though the parent's `nr_throttled` counts only its own limit). Rejected packets likewise also bump a named nftables counter that outlives per-sandbox chains. Two readings need care: a veth is a mirror (the sandbox's sent bytes are the host end's received bytes), and with `memory.oom.group`, `oom_kill` counts processes, not sandboxes.

15 Startup performance

For an agent, a sandbox start sits on the critical path of every tool call, so start latency was measured and then optimized. **Method:** the v2 harness times a one-shot `true` from immediately before process spawn until the CLI's `waitpid` returns: runtime setup, the command and synchronous cleanup through CLI exit, with the image local. Kernel-deferred destruction, such as a veth tied to a dying namespace, can finish after the timer. Every measured round contains every runtime in seeded, randomized Latin-square order. The primary campaign used 10 warm-up and 100 measured rounds at a defined 100 ms idle gap across five fresh VM boots; a 95% interval bootstraps the geometric mean of one median ratio per boot. A separate one-boot sweep measures 0, 10, 100, 1000 and 5000 ms gaps, because the absolute result changes with idle state. Every runtime starts the same rootfs contents under 512 MiB memory with no swap, 1 CPU and 128 PIDs.

The machine was an ARM64 Lima VM (4 vCPU, 4 GiB, Ubuntu 24.04, Linux 6.8) on an M2 MacBook Air under normal use. Raw JSONL and manifests record the config, rootfs and executable hashes, versions, kernel and boot ID. `true` is an overhead microbenchmark, not evidence that application code runs faster. `KILN_TRACE=1` prints each phase of a start on one `CLOCK_MONOTONIC` timeline for the host side and the sandbox's init.

Table 2: Primary v2 results for end-user stacks on one ARM VM with identical rootfs contents (`docs/benchmarks.md`). The 5 s values are 10-run p50s from one idle-gap sweep; ratios use five fresh boots; concurrency is three 64-client batches; memory is the median of three 0–100-sandbox fits.

	kiln	Docker	Podman + crun	nerdctl
5 s-idle `true`, no network (ms)	**14.88**	140.90	109.27	147.49
100 ms ratio to kiln (5 boots)	**1.00**	25.84	17.49	27.64
64-way `true`, completions/s	**836.6**	6.9	15.0	10.5
Idle memory slope (KiB/sandbox)	**749**	5007	1203	13172
Shared daemon, no network	none	dockerd + containerd	none	containerd

Where the time went. In the earlier fixed-order 100 ms-gap series, the baseline took 28.9 ms (p50) without a network and 87.9 ms with one. Almost all of it was two things:

- *RCU grace periods* (5–20 ms each on this VM). Writing a PID to `cgroup.procs` migrates a running process under a per-CPU rwsem whose writer waits for a grace period (8.6 ms in the trace). Closing an nftables netlink socket after a commit that deleted anything waits for nf_tables' deferred destructor (`netlink_release` → `flush_work`), 10–20 ms, and a networked run paid it up to four times (a base-ruleset "repair" that rewrote a set on every run, and three teardown `nft` processes). Deleting the host end of the veth unregistered it synchronously, another 10–15 ms.
- *Helper processes.* A cold `nft` costs 3.9 ms before doing anything (exec plus a dozen shared libraries). A networked start ran 8 `nft`, 2 `ip` and 1 `nsenter`; even `--net none` ran `nsenter` and `ip` to bring loopback up.

A back-to-back benchmark hid most of this (2.7 ms per start): a grace period just completed by the previous run makes the next migration cheap. Setting `rcu_expedited` confirmed the diagnosis: it cut the old build from 29.0 to 17.0 ms and changes nothing for the new one (13.2 vs 13.3 ms).

Fixes. `clone3(CLONE_INTO_CGROUP)` (no migration, as crun does); rtnetlink instead of `ip` and `nsenter`; one `nft` per networked start, committing base-ruleset repairs only on drift; teardown deletions as one nfnetlink batch whose socket a short-lived child closes, so kiln does not wait for the deferred freeing; and the veth left to the dying namespace. The phase trace left mostly namespace creation (1.6 ms), the child's mounts (2.5 ms), starting the binary and about 5 ms between the command exiting and the host seeing the sandbox gone; a networked start adds one `nft` process (about 6 ms) and the veth. The v2 sweep later measured kiln at 2.78 ms back to back and 14.88 ms after 5 s idle, with bare crun at 2.58 and 14.80 ms: the optimization reached the low-level floor, but the floor itself depends on idle state.

Table 2 compares complete user-facing stacks. The low-level floor was bare crun: 14.80 ms at the 5 s gap and 827.0 completions/s at 64 clients, essentially kiln's result. Bare runc took 34.81 ms; youki 30.96 ms. gVisor runsc took 67.45 ms and 13.7 MiB per idle sandbox, but belongs to a stronger isolation tier. youki timed out 22 commands under concurrency; every other runtime completed all commands.

The corrected network pair was rerun separately. At 64 clients kiln averaged 114.4 completions/s against 10.6 for Podman with crun; their median idle-memory slopes were 1101 and 1471 KiB per sandbox, with overlapping three-fit ranges (550–1444 and 914–1844 KiB). The first concurrency attempt found one real kiln failure in 351 invocations: a dying network namespace removed a reused veth between an `EEXIST` response and stale-link deletion. That attempt is preserved but excluded. The fix treats only the resulting `ENODEV` as already-cleaned state, adds a ten-wave 160-start concurrent stress check, and the fresh-boot replacement completed all 702 invocations across the pair. This is one-boot exploratory evidence, not a capacity or memory-ranking claim.

The stacks still do different work on different storage paths. kiln creates a user namespace, seccomp, an overlay on a quota'd XFS loop device and PID 1; Docker adds its daemon path (CLI → dockerd → containerd → shim → runc) and an ext4-backed overlay snapshot; Podman uses conmon; nerdctl crosses containerd. Bare runtimes receive a prepared bundle on ext4 with no seccomp or overlay. This is a product-level comparison, not attribution to one component.

What the numbers do not show. The five-boot intervals apply only to ratios at the defined 100 ms gap. The 5 s sweep, concurrency and memory fits each use one boot and remain exploratory. Concurrency is a barrier-released client-load test, not perfectly simultaneous kernel starts: at level 64, kiln without networking spread 15.7–20.4 ms, and the widest original batch (Docker with networking) spread 7.87 s. Representative shell, Python, Node, filesystem, DNS and HTTP one-shot workloads used three fresh-boot sessions per workload; the Docker/kiln ratio narrowed from 21.36× for the tiny shell command to 4.67× for Python, 3.81× for Node and 2.15× for HTTP, as fixed overhead was amortized. DNS and HTTP include public-network variance. The earlier warm-exec series (kiln 3.5 ms, crun 9.7, runc 24.1, Docker 53.7) was not repeated under v2. No CPU difference was observed in the seven-run steady-state test; its I/O result is inconclusive because storage differed. Firecracker and Kata could not run because the VM exposed no `/dev/kvm`. Next measurements are a second host (the intended Mac Mini), x86-64 bare metal, replicated concurrency

Table 3: Classes of issues found, the fix, and the design lesson.

Class	Finding and fix	Lesson
A missing namespace	No IPC namespace: sandboxes shared System V IPC objects. Fix: `CLONE_NEWIPC`.	Enumerate every kernel-global resource. User namespaces do not separate sandboxes that share one ID range.
Inherited handles	A descriptor open in kiln's caller was writable inside a `--read-only` sandbox. Fix: close all others first.	Path checks do not revoke handles. Same class as runc's CVE-2024-21626.
State that outlives its owner	Reused IPs let a new sandbox continue the previous one's conntrack flows; `add chain` kept a crashed run's rules. Fix: policy on every packet; flush after add.	Anything keyed by a reusable identifier (IP, inode number) must be reset, not assumed fresh. The quota code follows the same rule.
Input at a trust boundary	`--image` accepted `../..`; size flags could overflow. Fix: plain names only; checked multiplication.	Validate CLI input as if it came from an agent, because eventually it will.
Lifecycle edge cases	Fix: seccomp `TSYNC`; `PDEATHSIG` set after the credential change that clears it; Landlock ABI 3 required.	Kernel features have ordering and version preconditions.
Second review (security-focused model, after the metrics layer)		
Unbounded host-side work	The DNS forwarder started a thread per query, so sandboxes could make it create threads without limit; the metrics server handled one connection at a time, so one slow client stalled every scrape. Fix: fixed worker pools, bounded queues.	Anything a sandbox can talk to is part of the isolation boundary. Its resource use must be bounded independently of what sandboxes do.
A limit on one dimension only	The disk quota capped blocks but not inodes; empty files cost almost no blocks, so one sandbox could exhaust the filesystem's inodes. Fix: one inode per 16 KiB of quota.	Limit every dimension of a resource: bytes *and* count, as `memory.max` and `pids.max` do for memory and processes.
Untrusted bytes in a trusted channel	DNS query names (arbitrary bytes chosen by the sandbox) were written to the journal unescaped. Fix: escape control characters.	Logs are an output that operators trust; escape attacker-controlled data before it reaches them.
Insecure defaults	Grafana kept its default administrator password. Fix: generate one at install time.	The observability stack ships as part of the system; its defaults are the system's defaults.
Crash leftovers	A killed supervisor's IP lease stayed taken forever. Fix: reclaim it once cgroup, veth, map entry and supervisor are all gone.	Reclaim on agreement of several independent signals; any one of them is absent during a healthy setup.
Existence is not correctness	The base ruleset was created only if the table was missing; a partial table stayed broken. Fix: reconcile every shared object.	Converge on a desired state, as a controller does, instead of checking that something exists.
Supply chain	The image was trusted on a checksum from the same mirror and unpacked in place. Fix: pinned-key signature check, strict name, staged `renameat2` swap.	A checksum proves integrity, not origin. Anything sandboxes start from should change atomically.
Shared kernel tables	One sandbox could fill the host-wide conntrack table. Fix: at most 256 tracked flows per sandbox IP, with a rejection counter.	Per-sandbox limits apply to kernel tables too, not only to cgroup resources.
Found while verifying the fixes		
State that outlives its owner, again	The per-IP conntrack count outlived the sandbox, locking out the next one on the same IP until old flows expired. Fix: reset it at setup and teardown.	The first review's lesson, in a new place: state keyed by a reused IP is inherited. A fix can reintroduce the class it guards against.

and memory, matched storage, larger images and KVM-based isolation. The committed traces reproduce the current analysis. A close measurement rerun also needs the rootfs tarballs identified by hash in the manifests, which should be published as release artifacts first. The original resolved Lima instance configuration was not archived, so a byte-for-byte environment replay is not possible.

16 Adversarial model review: what it found and taught

kiln went through two model-assisted adversarial review rounds. After networking landed, four independent model reviewers attacked it with the goal of breaking isolation. After the metrics layer, a separate security-focused model examined the whole system, including the host-side services. These rounds were useful engineering review, not a professional security audit. The second found five concrete issues and four it rated theoretical; all nine are fixed. The focused regression suite exercises the sandbox-visible security and lifecycle invariants, while service-only changes are also checked in their code and configuration. Table 3 summarizes both rounds by class of issue, not by individual reproduction.

In the first review, two issues were documented rather than fixed: Landlock does not cover metadata, and DNS to the host's resolver was reachable. The second was later closed by the forwarder. Separately, testing found the `O_DIRECT` OOM of §13. The meta-lesson of the first round: each layer was correct in isolation; the bugs were at the seams, in what one mechanism assumed another had already handled. The second round's lesson: isolating sandboxes moves the attack surface onto the shared services and state that serve them.

The last row was caught when a plain `--net full` sandbox got "Host is unreachable" right after the suite ran; the suite now checks for it.

17 Known limitations and future work

Limitations today.

- *Reconcile is manual and partial.* `kiln gc` cleans up persistent sandboxes whose shim died, and stale leases are reclaimed at allocation. A one-shot run whose supervisor is `SIGKILL`ed still leaves its cgroup and run directory (and so its lease), and nothing reconciles automatically at boot.
- *Exec is serial* (one command at a time) and allocates no TTY.
- *Conntrack accounting restarts per sandbox.* Resetting the per-IP count on reuse means rapidly recycled sandboxes can together briefly exceed 256 tracked flows per IP until old entries expire.
- *One shared ID range.* All sandboxes map to the same host IDs, so they are not separated from each other by the user namespace. Idmapped mounts (`mount_setattr` with `MOUNT_ATTR_IDMAP`) would allow a range per sandbox without copying the image.
- *Allowlist drift.* `--allow` hostnames are resolved once at start; CDN addresses change. The forwarder also allows subdomains of an allowed name whose addresses the firewall does not. Next step: the forwarder adds the addresses it resolves to the sandbox's nftables set.
- *Also:* DNS tunnelling under `--net full` is logged but allowed; the seccomp denylist is weaker than an allowlist; all sandboxes share the host kernel; IPv4 only; I/O weights are accounted but not enforced on this VM.

Future work. The build order continues with: (1) **kilnd**, an API daemon over the existing create/exec/rm lifecycle: a state machine with run records in SQLite and automatic reconcile at startup by scanning `/sys/fs/cgroup/kiln/` and `/var/lib/kiln/runs/`. The shims let sandboxes survive a daemon crash; a backend trait leaves room for a Firecracker microVM backend. (2) **The agent harness**: an agent loop with memory and tools (`exec`, `read_file`, `write_file`, `diff`) talking to kilnd over a Unix socket, with the upper layer as audit log and snapshot base. (3) **A pool of pre-created sandboxes**, so a full lifecycle becomes an exec (3.5 ms in the legacy exec series). (4) **A Kubernetes operator**: a `Sandbox` custom resource on k3s, with kilnd as a DaemonSet.

18 Conclusion

A container is a set of independent kernel mechanisms, and most of the engineering is in their ordering and seams: limits before the first instruction, the user namespace last, descriptors closed first, reused identifiers reset, and, for speed, no needless waits on the kernel's grace periods.

References

[1] Linux manual pages: *namespaces*(7), *user_namespaces*(7), *pid_namespaces*(7), *cgroup_namespaces*(7), *clone*(2), *pivot_root*(2), *seccomp*(2), *landlock*(7), *quotactl*(2), *xfs_quota*(8).

[2] Linux kernel documentation: *Control Group v2* (`admin-guide/cgroup-v2.rst`); *Overlay Filesystem* (`filesystems/overlayfs.rst`); *Landlock* (`userspace-api/landlock.rst`).

[3] CVE-2024-21626: runc file descriptor leak ("Leaky Vessels"), GitHub advisory GHSA-xr7r-f8xq-vfvv, January 2024.

[4] Docker documentation: *Seccomp security profiles for Docker*; *Isolate containers with a user namespace* (userns-remap); *tini*, `github.com/krallin/tini`.

[5] *nftables wiki* (verdict maps, NAT in inet tables, named counters); Prometheus documentation, *Exposition formats*.

A Common questions and answers

Positioning

Q: How does kiln compare with Docker and runc?

Docker is a stack: a daemon, containerd, a per-container shim, and runc, which does the actual namespace, cgroup and mount setup from an OCI bundle. kiln covers roughly runc's job plus the parts of Docker that matter for sandboxing (image storage with overlayfs, bridge networking with NAT, disk quotas, an egress policy, metrics) in one binary. It has no OCI image or runtime-spec support and no image distribution. In one idle-gap sweep, after 5 s idle, its one-shot `true` lifecycle took 14.9 ms (p50) against 140.9 ms for the Docker CLI stack as configured. Across five boots at a 100 ms gap, the paired geometric mean of Docker/kiln per-boot median ratios was 25.84× (95% bootstrap interval 23.82–27.82×). Part of that gap is the daemon path, though the two also differ in storage and isolation.

Q: And with gVisor and Firecracker?

All three answer "how much of the host kernel does untrusted code reach?" kiln, like runc, lets the sandbox call the host kernel directly and narrows that with namespaces, seccomp and Landlock. gVisor intercepts syscalls and serves most of them from a user-space kernel, so far less host kernel code is reachable, at a cost in syscall-heavy workloads. Firecracker runs a guest kernel in a KVM microVM: the strongest boundary of the three, at the cost of a VM per sandbox. kiln's planned backend trait is meant to let a microVM backend sit behind the same API.

Q: Why Rust?

A runtime runs as root and parses untrusted input (DNS packets, HTTP requests), so memory safety matters. `unsafe` is confined to small, commented blocks around raw syscalls and `ioctl`s, which makes the dangerous parts easy to audit; `#[repr(C)]` structs have their sizes checked at compile time; and `Drop` gives every kernel object (cgroup, nft chain, lease) cleanup on every exit path. Rust also has no runtime threads, so `clone` and `fork` behave as in C. Go's multi-threaded runtime is why runc does its namespace setup in a small C shim before Go starts.

Q: Persistent sandboxes exist. Why is there still no central sandbox-manager daemon?

A sandbox that outlives a command needs a long-lived owner, but that owner does not have to be central: each sandbox gets its own shim, so there is no single process whose crash takes every sandbox down, and state still lives only in the kernel and the run directory. The fixed DNS service used for networked sandboxes and the optional metrics service do not own sandbox lifecycle. A manager daemon is the next step for what a shim cannot do: an API for the harness, run records, automatic reconcile at startup, and a pool of pre-created sandboxes. Because run IDs name cgroups and run directories, it can rebuild its view by scanning them, as `kiln gc` already does.

Mechanisms

Q: How do you guarantee limits apply before the command runs?

`clone3` with `CLONE_INTO_CGROUP` creates the child already inside its cgroup, so it has no instruction outside its limits. The older way, writing its PID to `cgroup.procs` before releasing it from a pipe, is also correct but migrates a running process, which waited 5–15 ms for an RCU grace period on every cold start.

Q: Why is the user namespace created last?

Each namespace is owned by the user namespace current at its creation, and capabilities apply only to namespaces one's user namespace owns. Creating it last means the mount, network, UTS, IPC, PID and cgroup namespaces belong to the host, so sandbox root's capabilities do not reach them.

Q: How does `kiln exec` get into a running sandbox?

It doesn't enter; it asks. PID 1 holds a Unix socket the shim opened before `pivot_root`. `kiln exec` sends the arguments and passes its redirected stdin, stdout and stderr with `SCM_RIGHTS`; terminal descriptors are refused until kiln has a controlled PTY mode. PID 1 forks the command onto those descriptors and returns its exit code. The command inherits every namespace, limit, seccomp filter and Landlock rule from PID 1, so there is no second, `setns`-based setup path to keep in sync.

Q: Why does the command run as PID 2?

PID 1 of a namespace ignores signals from inside it unless it installed a handler, and inherits all orphans. kiln-init forwards termination signals and reaps zombies, so `timeout`, `kill` and long-running shells behave normally.

Q: Why overlayfs instead of copying the image for each run?

Copying costs time and disk per run; an overlay mount is constant-time, every sandbox shares the image's page cache, and the upper layer is an exact diff of the run, which is both an audit log and the basis for snapshots.

Q: Why cgroup v2?

One hierarchy instead of one per controller, and the features kiln relies on exist only there: `memory.oom.group`, `cgroup.kill`, pressure stall information, iocost, and cgroup namespaces that show a sandbox only its own subtree. Ubuntu 24.04 uses v2 by default.

Q: What does an agent see when a sandbox runs out of memory?

`memory.oom.group` kills every process in the sandbox at once, so the run ends with exit code 137 (128 + SIGKILL), and the supervisor reads `oom_kill` from `memory.events` and says so. Writing large files does not cause this: page cache is charged but reclaimable.

Q: Why XFS project quotas rather than limiting by UID?

All sandboxes share one host ID range, so a UID quota could not tell them apart. A project quota limits a directory tree, and `PROJINHERIT` tags everything the sandbox creates. The run directory's inode number serves as a free, collision-free project ID, and each project gets both a block and an inode limit.

Q: Why reject instead of drop, and why isn't "established" accepted first?

Reject makes blocked connections fail fast, which an agent loop needs. Outbound packets are always checked against the sandbox's current policy because IPs are reused and conntrack entries outlive sandboxes; "established" is trusted only for replies coming back in.

Q: How do you stop data exfiltration through DNS?

Sandboxes can reach only kiln's forwarder. With `--allow`, it resolves only the allowed names and their subdomains and refuses the rest, so an attacker's domain never reaches a resolver. With `--net full`, queries are logged but allowed.

Q: How do you know the image hasn't been tampered with?

The fetch script accepts only a strictly formed file name, checks the SHA-256, and verifies Alpine's detached signature against a release key pinned by fingerprint in the repository. It unpacks into a staging directory and swaps it in with `renameat2(RENAME_EXCHANGE)`, so no sandbox ever starts from a partly installed image.

Q: Why is the DNS forwarder a separate service?

It listens on one shared address, 10.88.0.1:53, for every sandbox, and it keeps no per-run state: each run only writes a policy file named after its IP. A single long-lived service under systemd, with its own task and memory caps, is simpler and easier to bound than a forwarder per run.

Operations

Q: How do metrics survive short-lived runs?

A run that ends between two 5 s scrapes takes its cgroup and nftables chain with it. The exporter therefore also reads the parent cgroup, whose `memory.events` and `cpu.stat` are hierarchical and keep counting after children are removed, and a named nftables counter that every reject increments and that lives as long as the table. Totals since boot stay exact even when no scrape ever saw the run.

Q: What happens if the supervisor is killed with SIGKILL?

The sandbox dies too (`PR_SET_PDEATHSIG`). For a persistent sandbox, `kiln gc` then removes its cgroup, network state and run directory. For a one-shot run, the cgroup and run directory remain until cleaned up by hand; automatic reconcile is daemon work. Catchable signals (Ctrl-C, `kill`) are consumed by `sigwait` and lead to a normal cleanup.

Q: How fast does a sandbox start?

It depends on idle state. In a one-boot sweep, a one-shot `true` (setup, command and teardown) took 2.78 ms p50 back to back and 14.88 ms after 5 s idle; bare crun took 2.58 and 14.80 ms, and Docker 141.54 and 140.90 ms. Across five fresh boots at a defined 100 ms gap, the paired geometric mean of Docker/kiln per-boot median ratios was 25.84× (95% interval 23.82–27.82×). The original build took 28.9 and 87.9 ms in the legacy no-network/network series; removing RCU waits and helper processes produced the current path. A command in an existing sandbox took 3.5 ms in an earlier one-session exec benchmark.

Q: How would you test it?

Today, each limit was verified by triggering its failure mode on purpose (OOM kill, fork bomb, full disk, rejected packet), `scripts/metrics-demo.sh` exercises the whole stack, and `scripts/review-regressions.sh` checks the sandbox-visible second-review fixes, the persistent lifecycle and that no state is left behind. It already caught one regression (the inherited conntrack count). Next: run it in a VM in CI; turn the first review's findings into tests too; unit-test the seccomp program and size parsing; and fuzz the DNS question parser, the one piece that parses sandbox-controlled bytes on the host.

Q: How would this run on many hosts?

A daemon per node owns local sandbox state and reconciles from the kernel and disk. On k3s, a Go operator with a `Sandbox` custom resource places sandboxes on nodes, with the daemon deployed as a DaemonSet. Node-local state (bridges, leases, quotas) stays node-local; the operator only schedules, using per-node capacity it can read from the same cgroup metrics.

Q: Is this safe for hostile code?

It is defense in depth on a shared kernel: user namespace, seccomp, Landlock, network isolation and resource limits. A kernel bug reachable through an allowed syscall would still be an escape, and all sandboxes share one host ID range. That is why a microVM backend, a seccomp allowlist and per-sandbox ID ranges are on the roadmap.

Q: What would you do differently?

Benchmark cold rather than back to back from the first day (back to back hid most of the old start time); use pidfds rather than PIDs; use idmapped mounts instead of rewriting image ownership, which would also give each sandbox its own ID range; begin with a seccomp allowlist; and write the regression tests alongside each layer rather than after the reviews.